

AGENTS

AGENTS.md - LLMProvider Module

Base agent rules: HelixConstitution/AGENTS.md — READ IT FIRST. All rules in HelixConstitution/AGENTS.md apply unconditionally. Rules below extend them and MUST NOT weaken any inherited clause.

Module Overview

`digital.vasic.llmprovider` is a generic, reusable Go module providing LLM provider abstractions and utilities. It defines the core `LLMProvider` interface and common patterns for building LLM provider implementations, including circuit breakers, health monitoring, retry logic, and lazy loading. The module is designed for AI/LLM applications that need to integrate multiple LLM providers with fault tolerance and observability.

Module path: `digital.vasic.llmprovider` **Go version:** 1.25.3+ **Dependencies:** `digital.vasic.models`, `github.com/sirupsen/logrus` **Test Dependencies:** `github.com/stretchr/testify`

Package Responsibilities

Package	Path	Responsibility
<code>llmprovider</code>	<code>./</code>	Core types: <code>LLMProvider</code> interface, circuit breaker, health monitor, retry config, lazy provider, and associated utilities. This is the only package.

Dependency Graph

`digital.vasic.models`

↓

digital.vasic.llmprovider
↓
github.com/sirupsen/logrus

Key Files

File	Purpose
provider.go	LLMProvider interface definition with Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig methods
circuit_breaker.go	Circuit breaker implementation (503 lines) with closed/open/half-open states, failure counting, timeout
health_monitor.go	Health monitoring (430 lines) with configurable thresholds, intervals, status transitions
retry.go	Retry logic (226 lines) with exponential backoff, jitter, HTTP status code detection
types.go	Empty file (types moved to models module)
circuit_breaker_test.go	Circuit breaker tests (585 lines)
health_monitor_test.go	Health monitor tests
retry_test.go	Retry logic tests
types_test.go	Empty test file
go.mod	Module definition and dependencies
CLAUDE.md	AI coding assistant instructions
README.md	User-facing documentation with quick start

Agent Coordination Guide

Division of Work

When multiple agents work on this module simultaneously, divide work by component categories:

1. **Interface Agent** -- Owns LLMProvider interface definition and related methods. Changes affect all provider implementations.
2. **Circuit Breaker Agent** -- Owns circuit breaker implementation and CircuitBreakerManager. Affects fault tolerance.
3. **Health Monitor Agent** -- Owns health monitoring, thresholds, status transitions.

4. **Retry Logic Agent** -- Owns retry configuration, exponential backoff, HTTP status detection.
5. **Lazy Provider Agent** -- Owns lazy initialization pattern and deferred provider creation.

Coordination Rules

- **LLMProvider interface changes** require coordination with all agents as this is the foundational interface.
- **Circuit breaker behavior changes** affect fault tolerance across all providers.
- **Health monitor thresholds** affect provider health detection and automatic failover.
- **Retry logic changes** affect error recovery and rate limiting handling.
- **Lazy provider changes** affect initialization patterns and startup performance.

Safe Parallel Changes

These changes can be made simultaneously without coordination:

- Adding new methods to existing structs (if they don't affect interface)
- Adding new configuration options with sensible defaults
- Adding new helper functions
- Updating documentation
- Adding new test cases
- Improving error messages or logging

Changes Requiring Coordination

- Modifying LLMProvider interface method signatures (breaks all implementations)
- Changing circuit breaker state machine logic (affects fault tolerance)
- Modifying health monitor status transition thresholds (affects provider health detection)
- Changing retry exponential backoff formula (affects error recovery timing)
- Modifying lazy provider initialization semantics (affects startup behavior)

Build and Test Commands

```
# Build all packages
go build ./...
```

```
# Run all tests with race detection
```

```
go test ./... -count=1 -race

# Run unit tests only (short mode)
go test ./... -short

# Run a specific test
go test -v -run TestCircuitBreaker ./...

# Format code
gofmt -w .

# Vet code
go vet ./...

# Check dependencies
go mod tidy
go mod verify
```

Commit Conventions

Follow Conventional Commits with llmprovider scope:

```
feat(llmprovider): add new method to LLMProvider interface for
batch processing
feat(circuit): add configurable failure threshold to circuit
breaker
feat(health): add degraded state to health monitor
feat(retry): add jitter to exponential backoff
fix(circuit): correct race condition in state transition
test(health): add edge case tests for health monitor thresholds
docs(llmprovider): update API reference for new methods
refactor(retry): extract common backoff calculation functions
```

Thread Safety and Concurrency

- CircuitBreaker uses `sync.RWMutex` for thread-safe state management
- HealthMonitor uses atomic operations for status updates
- CircuitBreakerManager is thread-safe for concurrent provider management
- RetryConfig is immutable after creation
- LazyProvider uses `sync.Once` for thread-safe lazy initialization
- All exported methods are safe for concurrent use unless otherwise documented

Integration Notes

- This module depends on `digital.vasic.models` for `LLMRequest`, `LLMResponse`, `ProviderCapabilities` types
- Provider implementations should import this module and implement the `LLMProvider` interface
- The module is designed for zero-dependency provider implementations (implementations only need this module and models)
- Circuit breaker and health monitor can be composed around any `LLMProvider` implementation

Host Power Management — Hard Ban (CONST-033)

You may NOT, under any circumstance, generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition. This rule applies to:

- Every shell command you run via the Bash tool.
- Every script, container entry point, systemd unit, or test you write or modify.
- Every CLI suggestion, snippet, or example you emit.

Forbidden invocations (non-exhaustive — see CONST-033 in `CONSTITUTION.md` for the full list):

- `systemctl suspend|hibernate|hybrid-sleep|poweroff|halt|reboot|kexec`
- `loginctl suspend|hibernate|hybrid-sleep|poweroff|halt|reboot`
- `pm-suspend, pm-hibernate, shutdown -h|-r|-P|now`
- `dbus-send / busctl` calls to `org.freedesktop.login1.Manager.Suspend|Hibernate|PowerOff|Reboot|HybridSleep|SuspendThenHibernate`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to anything but `'nothing'` or `'blank'`

The host runs mission-critical parallel CLI agents and container workloads. Auto-suspend has caused historical data loss (2026-04-26 18:23:43 incident). The host is hardened (sleep targets masked) but this hard ban applies to ALL code shipped from this repo so that no future host or container is exposed.

Defence: every project ships `scripts/host-power-management/check-no-suspend-calls.sh` (static scanner) and `challenges/scripts/no_suspend_calls_challenge.sh` (challenge wrapper). Both **MUST** be wired into the project's CI / `run_all_challenges.sh`.

Full background: docs/HOST_POWER_MANAGEMENT.md and CONSTITUTION.md (CONST-033).

CONST-035 — Anti-Bluff Tests & Challenges (mandatory; inherits from root)

Tests and Challenges in this submodule **MUST** verify the product, not the LLM's mental model of the product. A test that passes when the feature is broken is worse than a missing test — it gives false confidence and lets defects ship to users. Functional probes at the protocol layer are mandatory:

- TCP-open is the FLOOR, not the ceiling. Postgres → execute `SELECT 1`. Redis → `PING` returns `PONG`. ChromaDB → `GET /api/v1/heartbeat` returns 200. MCP server → TCP connect + valid JSON-RPC handshake. HTTP gateway → real request, real response, non-empty body.
- Container Up is NOT application healthy. A `docker/podman ps` Up status only means PID 1 is running; the application may be crash-looping internally.
- No mocks/fakes outside unit tests (already CONST-030; CONST-035 raises the cost of a mock-driven false pass to the same severity as a regression).
- Re-verify after every change. Don't assume a previously-passing test still verifies the same scope after a refactor.
- Verification of CONST-035 itself: deliberately break the feature (e.g. kill <service>, swap a password). The test **MUST** fail. If it still passes, the test is non-conformant and **MUST** be tightened.

CONST-033 clarification — distinguishing host events from sluggishness

Heavy container builds (BuildKit pulling many GB of layers, parallel podman/docker compose-up across many services) can make the host **appear** unresponsive — high load average, slow SSH, watchers timing out. **This is NOT a CONST-033 violation.** Suspend / hibernate / logout are categorically different events. Distinguish via:

- uptime — recent boot? if so, the host actually rebooted.
- `loginctl list-sessions` — session(s) still active? if yes, no logout.

- `journalctl ... | grep -i 'will suspend\|hibernate'` — zero broadcasts since the CONST-033 fix means no suspend ever happened.
- `dmesg | grep -i 'killed process\|out of memory'` — OOM kills are also NOT host-power events; they're memory-pressure-induced and require their own separate fix (lower per-container memory limits, reduce parallelism).

A sluggish host under build pressure recovers when the build finishes; a suspended host requires explicit `unsuspend` (and CONST-033 should make that impossible by hardening `IdleAction=ignore + HandleSuspendKey=ignore + masked sleep.target, suspend.target, hibernate.target, hybrid-sleep.target`).

If you observe what looks like a suspend during heavy builds, the correct first action is **not** "edit CONST-033" but `bash challenges/scripts/host_no_auto_suspend_challenge.sh` to confirm the hardening is intact. If hardening is intact AND no suspend broadcast appears in journal, the perceived event was build-pressure sluggishness, not a power transition.

User-Session Termination — Hard Ban (CONST-036)

You may NOT, under any circumstance, generate or execute code that ends the currently-logged-in user's desktop session, kills their `user@<UID>.service` user manager, or indirectly forces them to manually log out / power off. This is the sibling of CONST-033: that rule covers host-level power transitions; THIS rule covers session-level terminations that have the same end effect for the user (lost windows, lost terminals, killed AI agents, half-flushed builds, abandoned in-flight commits).

Why this rule exists. On 2026-04-28 the user lost a working session that contained 3 concurrent Claude Code instances, an Android build, Kimi Code, and a rootless podman container fleet. The `user.slice` consumed 60.6 GiB peak / 5.2 GiB swap, the GUI became unresponsive, the user was forced to log out and then power off via the GNOME shell. The host could not auto-suspend (CONST-033 was in place and verified) and the kernel OOM killer never fired — but the user had to manually end the session anyway, because nothing prevented overlapping heavy workloads from saturating the slice. CONST-036 closes that loophole at both the source-code layer and the operational layer. See `docs/issues/fixed/SESSION_LOSS_2026-04-28.md` in the HelixAgent project.

Forbidden direct invocations (non-exhaustive):

- `loginctl terminate-user|terminate-session|kill-user|kill-session`

- `systemctl stop user@<UID> / systemctl kill user@<UID>`
- `gnome-session-quit`
- `pkill -KILL -u $USER / killall -u $USER`
- `dbus-send / busctl calls to org.gnome.SessionManager.Logout| Shutdown|Reboot`
- `echo X > /sys/power/state`
- `/usr/bin/poweroff, /usr/bin/reboot, /usr/bin/halt`

Indirect-pressure clauses:

1. Do not spawn parallel heavy workloads casually; check `free -h` first; keep `user.slice` under 70% of physical RAM.
2. Long-lived background subagents go in `system.slice`. Rootless podman containers die with the user manager.
3. Document AI-agent concurrency caps in `CLAUDE.md`.
4. Never script "log out and back in" recovery flows.

Defence: every project ships `scripts/host-power-management/check-no-session-termination-calls.sh` (static scanner) and `challenges/scripts/no_session_termination_calls_challenge.sh` (challenge wrapper). Both **MUST** be wired into the project's CI / `run_all_challenges.sh`.

CONST-035 — End-User Usability Mandate (2026-04-29 strengthening)

A test or Challenge that **PASSES** is a CLAIM that the tested behavior **works for the end user of the product**. The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work — buggy challenge wrappers masked failed assertions, scripts checked file existence without executing the file, "reachability" tests tolerated timeouts, contracts were honest in advertising but broken in dispatch. **This MUST NOT recur.**

Every PASS result **MUST** guarantee:

- a. Quality** — the feature behaves correctly under inputs an end user will send, including malformed input, edge cases, and concurrency that real workloads produce.
- b. Completion** — the feature is wired end-to-end from public API surface down to backing infrastructure, with no stub / placeholder / "wired lazily later" gaps that silently 503.
- c. Full usability** — a CLI agent / SDK consumer / direct curl client following the documented model IDs, request shapes, and endpoints **SUCCEEDS** without having to know which of N internal aliases the dispatcher actually accepts.

A passing test that doesn't certify all three is a **bluff** and MUST be tightened, or marked `t.Skip("...SKIP-OK: #<ticket>")` so absence of coverage is loud rather than silent.

Bluff taxonomy (each pattern observed in HelixAgent and now forbidden)

- **Wrapper bluff** — assertions PASS but the wrapper's exit-code logic is buggy, marking the run FAILED (or the inverse: assertions FAIL but the wrapper swallows them). Every aggregating wrapper MUST use a robust counter (`! grep -qs "|FAILED|" "$LOG"` style) — never inline arithmetic on a command that prints AND exits non-zero.
- **Contract bluff** — the system advertises a capability but rejects it in dispatch. Every advertised capability MUST be exercised by a test or Challenge that actually invokes it.
- **Structural bluff** — `check_file_exists "foo_test.go"` passes if the file is present but doesn't run the test or assert anything about its content. File-existence checks MUST be paired with at least one functional assertion.
- **Comment bluff** — a code comment promises a behavior the code doesn't actually have. Documentation written before / about code MUST be re-verified against the code on every change touching the documented function.
- **Skip bluff** — `t.Skip("not running yet")` without a `SKIP-OK: #<ticket>` marker silently passes. Every skip needs the marker; CI fails on bare skips.

The taxonomy is illustrative, not exhaustive. Every Challenge or test added going forward MUST pass an honest self-review against this taxonomy before being committed.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature

works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

Verification commands

Run before claiming a fix is complete:

```
bash scripts/anti-bluff/bluff-scanner.sh --mode all
bash yole-challenges/scripts/anchor_manifest_challenge.sh
bash yole-challenges/scripts/mutation_ratchet_challenge.sh
```

All three must PASS. Pre-existing bluff hits are tracked in yole-challenges/baselines/bluff-baseline.txt; do not extend the baseline without an explicit justification comment.

Skip-marker convention: // SKIP-OK: #<ticket> (canonical), // ANTI-BLUFF-EXEMPT: <reason> (synonym).

Submodule Decoupling & Reusability — MANDATORY for ALL AI Agents

Applies to ALL CLI agents (Codex, Cursor, Gemini CLI, Copilot CLI, Claude Code, etc.) working in this repository.

This repository is **shared infrastructure** consumed by multiple independent consumer projects. The value of this repository depends on staying fully decoupled and reusable.

Hard rules:

- DO NOT hardcode any specific consumer project's name, platform list, paths, version strings, or release-naming conventions.
- DO NOT import / reference any consumer-project namespace.
- DO NOT embed consumer-project-specific governance, branding, or rule numbering in CONSTITUTION.md / CLAUDE.md / AGENTS.md.
- DO assume $N \geq 2$ unrelated consumer projects exist.

Cross-project rules **MUST** be phrased generically ("every consuming project's full platform matrix"), never with a specific consumer's matrix hardcoded.

Article XI §11.9 — Anti-Bluff Forensic Anchor (cascaded from parent CONSTITUTION.md)

Verbatim user mandate (2026-04-29, reasserted multiple times across 2026-05): *"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"*

Operative rule: **The bar for shipping is not "tests pass" but "users can use the feature."** Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md alongside CONST-047 — see the parent repository's CONSTITUTION.md for the full text.

CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

Verbatim user mandate (2026-05-15): *"Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!"*

No feature / functionality / flow / use case / edge case / service / application on any supported platform of HelixCode may be considered deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture (CONST-035) with captured runtime evidence; (2) proof of working capability end-to-end on target topology (no mocks beyond unit tests — see CONST-050); (3) implementation matches documented promise; (4) no open issues/bugs surfaced — cross-checked against §11.4.15 / §11.4.16

trackers; (5) full documentation in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation).

Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status) regenerated as part of the release-gate sweep. Gaps tracked per §11.4.15 (UNCONFIRMED: / PENDING_FORENSICS: / OPERATOR-BLOCKED: with §11.4.21 audit) — rows that quietly omit a platform are CONST-048 violations.

Cascade requirement: This anchor (verbatim or by CONST-048 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.25 for the full mandate.

CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

Verbatim user mandate (2026-05-15): "Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Before ANY modification to constitution/Constitution.md, constitution/CLAUDE.md, or constitution/AGENTS.md, the agent or operator MUST execute the following 7-step pipeline in order:

1. **Fetch + pull first** inside the constitution submodule worktree — every configured remote fetched, then `git pull --ff-only` (or `--rebase` if non-FF; NEVER `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization).
2. **Apply the change** with §11.4.17 classification + verbatim mandate quote.
3. **Validate before commit** — `meta_test_inheritance.sh` (or equivalent), no merge-conflict markers, cross-file consistency.
4. **Commit + push to ALL upstreams** — governance files only (NEVER `git add -A`); push to every configured remote. One-upstream commit = CONST-049 violation (also CONST-038/§6.W and §2.1).
5. **Conflict resolution** preserving union of governance content. Force-push to bypass conflicts is FORBIDDEN (CONST-043 / §9.2).

6. **Post-merge validation** — `git submodule update --remote --init` + re-run cascade verifier (CONST-047) confirming the new clause reaches every owned submodule.
7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer advanced to the new constitution HEAD in the SAME commit as cascade work.

Cascade requirement: This anchor (verbatim or by CONST-049 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a force-push without CONST-043 / §9.2 authorization. No escape hatch. See constitution submodule Constitution.md §11.4.26 for the full mandate.

CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

Verbatim user mandate (2026-05-15): *"Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — <https://github.com/vasic-digital/Challenges>). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — <https://github.com/HelixDevelopment/HelixQA>). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."*

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, TODO, FIXME, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources (*_test.go files invoked without the integration build tag; HelixCode/tests/unit/; etc.). Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented HelixCode system against

real infrastructure (real PostgreSQL, real Redis, real LLM endpoints, real containers, real captured devices). Production code (anything under HelixCode/cmd/, HelixCode/applications/, HelixCode/internal/<pkg>/<file>.go not ending _test.go) MUST NOT import from HelixCode/internal/mocks/.

(B) 100% test-type coverage. HelixCode's codebase MUST be covered by every supported test type the domain warrants:

- **Unit** — fast, isolated, mocks permitted per (A).
- **Integration** — multi-component, no mocks, real backing services.
- **End-to-end (E2E)** — full user-flow exercise on target topology.
- **Full automation** — orchestrated suites exercising every feature × platform combination (CONST-048 coverage ledger).
- **Security** — authn/authz boundaries, CONST-042 secret-leak scans, input-fuzzing, dependency-CVE scanning, threat-model verification.
- **DDoS** — request-flood resilience at advertised throughput tier.
- **Scaling** — horizontal + vertical scale behaviour under linear load growth.
- **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew).
- **Stress** — sustained load above advertised tier.
- **Performance** — latency / throughput / tail-latency invariants vs SLO baselines.
- **Benchmarking** — micro + macro suites with historical p95-drift detection.
- **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface.
- **UX** — flow-correctness + accessibility + i18n + visual-cue ordering (§11.4.23 composition).
- **Challenges** — vasic-digital/Challenges submodule (at ./Challenges/) fully incorporated; per-feature Challenge scripts with captured runtime evidence.
- **HelixQA** — HelixDevelopment/HelixQA submodule (at ./HelixQA/) fully incorporated; ALL written test banks executed; full autonomous QA sessions run as part of release gates with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047):

- Challenges — git@github.com:vasic-digital/Challenges.git — incorporated at ./Challenges/.
- HelixQA — git@github.com:HelixDevelopment/HelixQA.git — incorporated at ./HelixQA/.
- Any additional functionality submodules under vasic-digital/* / HelixDevelopment/* orgs that HelixCode depends on — incorporate rather than duplicate work the orgs already maintain.

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (CONST-049 step 7). Pointer drift = CONST-050 violation.

Cascade requirement: This anchor (verbatim or by CONST-050 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule Constitution.md §11.4.27 for the full mandate.

CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

Verbatim user mandate (2026-05-15): "All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!"

Three cooperating invariants apply to every HelixCode-owned submodule (those whose upstream origin lives under vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, or any subsequently authorised org):

(A) Equal-codebase. Every owned-by-us submodule is an **equal part** of HelixCode's codebase. The same engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to each owned submodule on equal basis. A round of work that improves only HelixCode's main while leaving an owned-submodule deficiency unaddressed is a CONST-051 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. The §11.4.25 / CONST-048 coverage ledger **MUST** list every owned submodule as an in-scope target.

(B) Decoupling / reusability. Owned submodules **MUST** remain fully decoupled from HelixCode (and any other consuming project). No HelixCode-specific context, hardcoded paths, hostnames, asset names, or runtime assumptions may be introduced into an owned submodule's source tree. When a submodule needs information from HelixCode, the honest path is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent's tree. Every owned submodule **MUST** be project-not-aware, fully reusable, modular, and completely testable as a standalone repository.

(C) Dependency-layout. Every dependency that an owned submodule consumes **MUST** be accessible from HelixCode's root at one of two canonical paths:

- `<repo_root>/<submodule_name>/` (flat layout — current HelixCode layout for Challenges, HelixQA, Containers, Security, etc.)
- `<repo_root>/submodules/<submodule_name>/` (grouped layout — alternate)

Nested own-org submodule chains are FORBIDDEN. A submodule **MUST NOT** have its own `.gitmodules` entries pulling in further owned-by-us repos. Every dependency required by submodule X is added to HelixCode's root at the canonical path; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer. Third-party submodules (not under our orgs) are exempt — they **MAY** appear at any depth.

The owned-org list is dynamically discoverable at any time via `gh org list` / `glab` CLIs or the orgs' public APIs.

Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) **MUST** appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer. No escape hatch. See constitution submodule `Constitution.md` §11.4.28 for the full mandate (audit gates, mutation pairs, workflow integration).

Amendment Process

Constitution amendments require:

1. Written proposal with rationale
2. Challenge demonstrating the need
3. 72-hour review period
4. Approval by project architect
5. Update to all submodule governance files

This Constitution is the supreme law of the HelixCode project. No code, test, or process may contradict it.

CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

Verbatim user mandate (2026-05-15): *"naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in our paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!"*

Every directory, submodule, and file in HelixCode MUST use lowercase snake_case names. Existing non-compliant names (HelixCode/, Challenges/, Containers/, HelixAgent/, HelixQA/, Security/, Github-Pages-Website/, Upstreams/, Dependencies/, etc.) MUST be renamed as part of the phased migration opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a CONST-052 violation of equal severity to the rename itself.

Common-sense exceptions (technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE the language root (submodule root follows our convention; subtree follows language convention); vendor/upstream third-party submodules keep upstream names; build artefacts (node_modules, __pycache__, .git, target, build, bin) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

Upstreams/ → upstreams/ transition: the constitution submodule's install_upstreams.sh (exported via .bashrc/.zshrc) supports BOTH Upstreams/ and upstreams/ directory layouts (commit 45d3678 of the constitution submodule); lowercase wins when both present.

Test coverage of renames (per CONST-050(B)): every rename batch ships with (i) regression test verifying every reference now resolves, (ii) full test-type matrix run post-rename, (iii) anti-bluff wire-evidence captured.

Phased execution per the operator's explicit instruction: comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. §11.4.20 subagent delegation for cross-cutting rename sweeps.

Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer. No escape hatch beyond the common-sense exceptions enumerated above. See constitution submodule Constitution.md §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

Verbatim user mandate (2026-05-15): "every project module, every Submodule, every service and application MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivative which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be committed every time we are preparing to execute commit! If any violation is detected it MUST be fixed before commit is executed!"

Every project module, owned-by-us submodule, service, and application MUST ship a proper .gitignore. Forbidden-from-version-control classes:

1. **Build artefacts:** /bin/, /build/, /dist/, /out/, target/, *.exe, *.dll, *.so, *.dylib, *.a, *.o, *.class, *.pyc, generator-produced files when the generator is committed.
2. **Cache files:** __pycache__/, .pytest_cache/, .mypy_cache/, .ruff_cache/, node_modules/, .next/, .cache/, .gradle/, .terraform/, language-server caches.
3. **Temp files:** *.tmp, *.swp, *~, .DS_Store, Thumbs.db, *.orig, *.rej.
4. **Sensitive-data files:** .env, .env.* (allow .env.example placeholder only — no real secrets even as examples), *.pem, *.key, *.crt, id_rsa*, id_ed25519*, .netrc, secrets/, api_keys.sh.
5. **Generated reports/logs:** *.log, coverage.out, htmlcov/, runtime captures unless reference assets.
6. **OS/IDE personal state:** .idea/, .history/, .vscode/ (except shared settings).

Anti-bluff invariant: .gitignore line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked *.log despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged + git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to .gitignore, scrub if already-tracked). Gate CM-GITIGNORE-PRECOMMIT-AUDIT + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a .env leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by CONST-053 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule Constitution.md §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

Verbatim user mandate (2026-05-15): *"We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to everyone."*

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}],` `transitive_handling.{recursive,conflict_resolution},` `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by CONST-054 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule Constitution.md §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

Verbatim user mandate (2026-05-15): *"Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!"*

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by CONST-055 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

Verbatim user mandate (2026-05-15): *"Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"*

Every clone / add of a Git repository under HelixCode MUST be followed by install_upstreams invocation from the repository's root IF its tree contains upstreams/ (or legacy Upstreams/ per CONST-052 transition) populated with *.sh recipe files. The utility (installed on operator's PATH via .bashrc/.zshrc; implementation in the constitution submodule's install_upstreams.sh — already supports BOTH directory names since constitution commit 45d3678) reads the recipe files, configures every declared upstream as a named git remote, and fans out origin push URLs.

Skipping the invocation when upstreams/ is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate CM-INSTALL-UPSTREAMS-ON-CLONE + paired mutation. Automation: the future incorporate-submodule per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: git remote -v | grep -c push reports expected count.

Cascade requirement: This anchor (verbatim or by CONST-056 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal ****Status:**** value, drawn from this 3-element closed map:

Item **Type:**	Closure **Status:** value
Bug	Fixed (→ Fixed.md)

Item **Type:**	Closure **Status:** value
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (generate_issues_summary.sh, generate_fixed_summary.sh, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a CONST-057 violation. Gate CM-CLOSURE-VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose ****Status:**** is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by CONST-057 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose ****Status:**** is Reopened MUST carry, within 8 non-blank lines of the heading, a ****Reopened-Details:**** line capturing four sub-facts:

- **By:** AI or User (source-of-truth observer who flipped the status). AI covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). User covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (YYYY-MM-DD).
- **Reason:** one-line cause classification — chosen from the closed vocabulary { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }. Other values permitted with explicit Reason: <free text> annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 /

§11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The `Issues_Summary.md` Status column MUST distinguish the four Reopened sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: Reopened (AI: test-failed) vs Reopened (User: manual-testing). Gate CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by CONST-058 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md import` or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit Lifted from `<project>` to constitution per §11.4.35 / Demoted from constitution to `<project>` per §11.4.35 commit-message annotation.

Gate CM-CANONICAL-ROOT-CLARITY verifies (a) consumer's CLAUDE.md opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no ## INHERITED FROM block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by CONST-059 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.35 for the full mandate.

CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

Verbatim user mandate (2026-05-15): *"Make sure that feed-back_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details."*

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If HEAD..@{u} is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

Anti-bluff invariant: the fetch+log check MUST produce captured evidence — the actual HEAD..@{u} output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

Cascade requirement: This anchor (verbatim or by CONST-060 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to

§11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule Constitution.md §11.4.37 for the full mandate.

Anti-Bluff End-User Quality Guarantee (Escalated via HelixConstitution)

Canonical authority: HelixConstitution/Constitution.md §7.1 + §11.4.

**Forensic anchor — verbatim operator mandate
(2026-04-28):**

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

Every PASS MUST carry positive runtime evidence. Consuming-project-specific requirements are defined by each consuming project's Constitution. This submodule's rules remain project-agnostic.

CONST-061: Pre-Force-Push Merge-First Mandate (cascaded from constitution submodule §11.4.41)

Verbatim user mandate (2026-05-17): *"make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!"*

Any force-push (`--force`, `--force-with-lease`, `+<ref>`, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per appropriate strategy for every non-empty HEAD..`<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<< \|^===== $\|^>>>>>>'` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs without prior fetch; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in `helix_qa` + containers governance files).

Verification artefact: every governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section capturing fetch output, per-remote divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate CM-FORCE-PUSH-MERGE-FIRST + paired mutation.

Cascade requirement: This anchor (verbatim or by CONST-061 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. See constitution submodule Constitution.md §11.4.41 for the full mandate.

CONST-068: Shell-script target-shell-parseability mandate (cascaded from constitution submodule §11.4.67)

Verbatim user mandate (2026-05-19): *"any issue we spot must be fixed, bash scripts as well if they are broken!" + "Make sure that this is mandatory rule!"*

Verbatim 2026-05-19 operator mandate: *"all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"*

Every committed shell script MUST be parseable by its target interpreter (sh -n for /bin/sh, bash -n for /bin/bash, etc.) AND MUST declare a shebang matching its actual syntax usage. Bash-only constructs (>(...), <(...), [[]], <<<, arrays, \${var^^}, etc.) used in scripts that may be invoked via sh script.sh MUST be wrapped in eval so the parser sees only a string (target shells like mksh parse the entire script before executing — runtime guards cannot save a parse-time rejection). Honest shebangs only: #!/bin/bash only if bash actually expected; #!/bin/sh requires POSIX-clean body. Fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51. Pre-build gate CM-SCRIPT-TARGET-SHELL-PARSEABLE runs sh -n on every in-scope script. No escape hatch — no --skip-parseability-check, --bash-only-script, --runtime-guard-suffices flag.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary sibling — canonical pair), §11.4.19 (atomic Issues→Fixed migration trigger + column-alignment), §11.4.23 (colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects Fixed (→ Fixed.md) / Implemented (→ Fixed.md) / Completed (→ Fixed.md) terminal values), §11.4.44 (revision header applies to Fixed_Summary.md), §12.10 (CONTINUATION.md resumption guarantee).

Pre-build gates: CM-FIXED-SUMMARY-SYNC (6 invariants — Fixed_Summary exists + HTML/PDF mtime ≥ md mtime + Fixed_Summary mtime ≥ Fixed mtime + generator + sync wrapper invokes generator) + CM-COVENANT-114-53-PROPAGATION (anchor literal across canonical files). Paired mutations strip the an-

chor literal AND move the generator aside AND backdate
Fixed_Summary mtime. No escape hatch — no --skip-fixed-
summary-sync, --issues-only, --summary-not-applicable flag.

Canonical authority: constitution submodule Constitution.md
§11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential Phase-chain. Each PWU has:
ATM-NNN identifier (§11.4.54), Issues.md entry
(§11.4.15+§11.4.16), file-scope manifest, §11.4.43 RED test,
source patch, pre-build gate, post-flash test, paired §1.1 meta-test
mutation, HelixQA Challenge bank entry, captured-evidence
directory (§11.4.5+§11.4.52).

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in
worktrees) → Stage 2 MERGE (serial conductor + §11.4.41 4-step
merge-first) → Stage 3 REBUILD+FLASH (parallel where hard-
ware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-
test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md
migration + README refresh). Stage 1 of round N+1 overlaps
with Stages 4-5 of round N.

Synchronization: 4-layer lock hierarchy (parent flock / per- sub-
module git / contention-path advisory locks for 10 forbidden cross-
PWU paths / per-PWU worktree). Disjoint-scope PWUs fully paral-
lel.

Anti-bluff merge-time enforcement (mandatory, all four): C1
§11.4.43 RED-test captured. C2 §1.1 paired meta-test mutation
FAILs the gate. C3 §11.4.50 3-iter (or 10-iter) deterministic-con-
sistency. C4 §11.4.5 captured-evidence per feature type.
Metadata-only / configuration-only / absence-of-error / grep-
without-runtime PASS REJECTED. HelixQA Challenge bank
coverage MANDATORY for every user- visible PWU.

**Phase 39.EX infrastructure gates (5 gates land the parallel
infrastructure itself):** CM-PWU-PARALLEL-VALIDATION-ORCHESTRATOR,
CM-PWU-HELIXQA-PER-DOMAIN-RUNNER, CM-PWU-WORKER-POOL-LOCKING, CM-
PWU-FILE-SCOPE-PARTITION, CM-PWU-AUTO-MERGE-GATE-6CONDITIONS. Each
ships a paired meta-test mutation per §1.1.

Pre-build gates CM-PWU-LOCK-HIERARCHY + CM-PWU-ANTI-BLUFF-COVERAGE

- CM-PWU-MERGE-QUEUE-DISCIPLINE + CM-PWU-PARALLEL-AGENT-LIMIT +
CM-COVENANT-114-58-PROPAGATION. Paired mutations cover each
gate. No escape hatch.

Canonical authority: constitution submodule [Constitution.md](#)
§11.4.58. Project-specific implementation reference: [docs/guides/](#)
[PARALLEL_DEVELOPMENT_METHODODOLOGY.md](#).

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized .html and .pdf siblings. Includes: project-root *.md, docs/**/*.md, scripts/**/*.*md (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their docs/**/*.*md, constitution/**/*.*md, owned HelixQA submodules' equivalents. Excludes: external/**, prebuilts/**, packages/modules/**, kernel-5.10/**, out/**, build/**, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via scripts/testing/sync_all_markdown_exports.sh (pandoc HTML + weasyprint PDF, timeout 60 per file, capped at 500 candidates). HTML + PDF mtime MUST be $\geq$ source .md mtime at all times.

Pre-build gates CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC + CM-COVENANT-114-65-PROPAGATION. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no --skip-md-exports, --no-pdf-only, --md-export-not-applicable flag.

Canonical authority: constitution submodule [Constitution.md](#)
§11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (AskUserQuestion on Claude Code) — NEVER free-text "what would you like?" for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate CM-COVENANT-114-66-PROPAGATION enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no --skip-ask, --silent-wait, --free-form-only flag.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): "any issue we spot must be fixed, bash scripts as well if they are broken!"

- "Make sure that this is mandatory rule!"

Every shell script that may be invoked under a target shell other than the one in its shebang **MUST** parse cleanly under that target shell. Forensic incident: device/rockchip/rk3588/tests/test_all_fixes.sh:114 used bash-only exec > >(tee -a "\$f") 2>&1 on a sh script.sh callsite — Android mksh parses the whole script BEFORE executing, so the runtime [-n "\${BASH_VERSION:-}"] guard could not save it. Fixed by wrapping in eval 'exec > >(tee ...) 2>&1' so the parser sees only a string.

Closed-set scope: every tracked .sh under device/rockchip/rk3588/tests/, scripts/, scripts/testing/ (and equivalent paths in owned submodules). OUT of scope: external/, prebuilts/, packages/modules/, kernel-5.10/, out/, build/, scripts/legacy/. Mandatory invariants: (1) every in-scope script parses under sh -n; (2) bash-only constructs (>(...), <(...), [[]], <<<, arrays, \${var^^}, etc.) **MUST** be wrapped in eval OR guarded by bash-only loading; (3) shebangs honest — #!/bin/bash only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate CM-SCRIPT-TARGET-SHELL-PARSEABLE runs sh -n on every in-scope script. Propagation gate CM-COVENANT-114-67-PROPAGATION enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside eval → parse gate FAILs; strip 11.4.67 literal → propagation gate FAILs. No escape hatch — no --skip-parseability-check, --bash-only-script, --runtime-guard-suffices flag.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

"THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! ... Solution MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! ... Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!"

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio "82/84 PASS" + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence` <description> <evidence_path> — the new canonical PASS helper. Verifies path exists AND non-empty; emits PASS: <description> [evidence: <path>].
- `ab_skip_with_reason` <description> <closed-set-reason> — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §11.1 meta-test mutations:

- CM-SINK-EVIDENCE-PER-FEATURE — walks tests for # §11.4.69 FEATURE: <class> annotation + verifies taxonomy probe + ab_pass_with_evidence use.
- CM-NO-FAIL-OPEN-SKIP — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE — pre-grace WARN, post-grace FAIL on bare ab_pass calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no --skip-evidence, --config-only-pass, --allow-fail-open-skip, --legacy-ab-pass-permitted flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate CM-COVENANT-114-69-PROPAGATION enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule Constitution.md §11.4.69.

Non-compliance is a release blocker regardless of context.

Cascade requirement: This anchor (verbatim or by CONST-068 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.67 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

Verbatim user mandate (2026-05-20): *"We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for*

the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!"

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from /proc/asound/.../status hw_ptr; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / <unreachable> / <N.E.> / <None> placeholders are NOT positive evidence; a missing-but-required sink is OPERATOR-BLOCKED (release-blocker), never SKIP, never PASS. No escape hatch — no --skip-sink-evidence, --allow-empty-codec, --sink-unreachable-is-pass, --metadata-only-suffices flag exists.

Cascade requirement: This anchor (verbatim or by §11.4.68 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule Constitution.md §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

Verbatim user mandate (2026-05-20): "Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!"

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per superpowers:subagent-driven-development. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — --inline-execution-required, --no-subagents, --monolithic-execution

are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by §11.4.70 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule Constitution.md §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

Verbatim user mandate (2026-05-20): *"before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brought in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all components and functionalities covered and work fully in anti-bluff manner!"*

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (git fetch --all --prune --tags, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto---ours/--theirs); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with git ls-remote post-push. No escape hatch — no --skip-fetch, --no-investigate, --fast-push, --trust-upstream flag.

Cascade requirement: This anchor (verbatim or by §11.4.71 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a

§11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule Constitution.md §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

Verbatim user mandate (2026-05-20): *"Make sure all fixes for audio are always top priority in main working stream!"*

The conductor (main working stream — Claude Code session, AI agent, or human operator) **MUST** treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) **MAY** run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by §11.4.72 reference) **MUST** appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule Constitution.md §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

Verbatim user mandate (2026-05-20): *"Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document."*

Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!"

Applies **only** when a project recognises a main specification document. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table Revision integer, bumped for every other change); (3) the metadata table MUST stay current (Revision, Last modified, Status summary, Fixed); (4) propagated copies of the rule MUST reference the active specification.V<primary>.md, not a stale archive; (5) on primary bump the old file moves to <spec-dir>/archive/ with Status: superseded. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by §11.4.73 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a release blocker when a project has a main spec and lets it drift.

Canonical authority: constitution submodule Constitution.md §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

Verbatim user mandate (2026-05-20): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules further! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!"

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — vasic-digital and

HelixDevelopment on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a Catalogue-Check: field (reuse <org/repo>@<sha> / extend <org/repo>@<sha> / no-match <date>). Classification: universal.

Cascade requirement: This anchor (verbatim or by §11.4.74 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. Canonical authority: constitution submodule Constitution.md §11.4.74 for the full mandate.

§IS LLMProvider §11.4.X anchor propagation (Phase 39.IM, 2026-05-23)

The 18 blocks below were missing from this file's prior revision while present in the sibling LLMOrchestrator. Cascade requirement per Constitution §11.4.X propagation gates. Byte-identical to LLMOrchestrator source. See docs/Fixed.md §IS.

§11.4.1 extension (Phase 33, 2026-05-05) — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason (undefined variable under set -u, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Per parent Constitution §11.4.1, every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

Non-compliance is a release blocker regardless of context.

§11.4.2 extension (Phase 34, 2026-05-06) — Recorded-evidence requirement. A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Bug #13 (VK Video on PRIMARY display while a passing test claimed playback PASS) demonstrated the gap exactly. Closing it requires the recording + analyzer infrastructure (Bug #14 — `dual_display_record.sh / action_timeline.sh / Go recording-analyzer / helixqa-bridge`). Per Constitution §11.4.2 every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

Non-compliance is a release blocker regardless of context.

§11.4.3 extension (Phase 34, 2026-05-06) — Per-device-topology test dispatch. Tests that depend on hardware topology (secondary HDMI present/absent, microphone present/absent, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test running the wrong variant for the actual topology and PASSing is a §11.4 PASS-bluff. Bug #18 (Lampa+TorrServe E2E) demonstrated the pattern: D1 (secondary HDMI) and D2 (primary only) get separate test variants behind a `dumpsys display-based` dispatcher. Per Constitution §11.4.3 every topology-touching test MUST have such a dispatcher OR explicit topology gates with SKIP-with-reason fallback.

Non-compliance is a release blocker regardless of context.

§11.4.4 extension (User mandate, 2026-05-06) — Test-interrupt-on-discovery + retest-from-clean-baseline. A test cycle that continues running past a freshly discovered defect is itself a §11.4 PASS-bluff: it produces "all green" summaries while the codebase under test is known-broken at the moment those greens were recorded. Phase 34.S' D1 demonstrated the violation when Bug #26 (hard-floor probe lifecycle) and Bug #27 (analyzer FAIL-bluff on non-video tests) were discovered mid-cycle and the cycle was allowed to continue, accumulating 13+ false-positive ANALYZER FAIL banners. Per Constitution §11.4.4 the moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop on both devices. **Then:** (1) fix at root cause per §11.4.1, (2) land validation/verification tests for the fix — pre-build gate AND on-device test AND paired meta-test mutation, (3) full rebuild via `scripts/build.sh` (regardless of whether the fix touched host script / Go binary / firmware — host-only fixes still get a full rebuild for retest baseline integrity), (4) re-flash D1 + D2, (5) repeat full `test_all_fixes.sh` from the beginning sequentially per §12.6, (6) end the cycle with `meta_test_false_positive_proof.sh` proving no gate is itself a bluff gate. Tests AND HelixQA Challenges are bound equally — Chal-

lenges that score PASS on a non-functional feature are the same class of defect as PASS-bluff unit tests; both must produce positive end-user evidence per §11.4.2 + §11.4.3.

Non-compliance is a release blocker regardless of context.

§11.4.4 expansion (User mandate, 2026-05-06) — Systematic debugging + four-layer test coverage + documentation + no-bluff certification. Augments the §11.4.4 base covenant with four non-negotiable additional requirements per the User mandate of 2026-05-06: (a) **Systematic debugging via superpowers skills.** Before applying any fix, run in-depth systematic debugging using the available superpowers:* skills (debugging, root-cause analysis, architectural-impact). Symptom patches are forbidden. The debugging output MUST identify root cause at source layer, blast radius across related tests/features/subsystems, and the regression-protection seam. (b) **Four-layer test coverage per fix.** Every fix lands with positive evidence in **every applicable layer**: pre-build gate (catches at source), post-build gate (catches in assembled image — proves bytes landed, cf. Fix #122 APK_LIB_MAP misroute), post-flash on-device test (fully automated, anti-bluff per §8.1, captured- evidence per §11.4.2, topology-dispatched per §11.4.3, orchestrator- wired in test_all_fixes.sh), HelixQA test bank entry (banks/atmosphere.yaml + per-feature additions), HelixQA full QA session coverage (Challenge-driven dispatch — bank entry without Challenge coverage is a §11.4 PASS-bluff), and meta-test paired mutation. Skipping a layer because "this fix only touches X" is forbidden. (c) **Documentation update for every fix.** Required: docs/Issues.md → docs/Fixed.md migration on closure, parent CLAUDE.md Applied Fixes Reference row, affected user-facing guides (docs/guides/*.md), affected diagrams/flowcharts/architecture docs, per-version docs/changelogs/<tag>.md entry. Documentation drift after a fix is itself a §11.4 violation. (d) **No-bluff certification per cycle.** Before tagging: meta_test_false_positive_proof.sh returns all gates green AND every gate's paired mutation FAILs (no bluff gates); docs/Issues.md open-set is empty or every entry explicitly classified out-of-scope-for-this-tag with operator sign-off (no known issues hidden); full suite returns zero new FAILs on either device (no working feature regressed); every gate has a paired mutation; every test produces positive evidence; every assertion catches its own negation (no error-prone or bluff-proof leftover).

Non-compliance is a release blocker regardless of context.

§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim, 2026-05-07):

"We MUST HAVE still analyzing of recorded materials and comprehensive validation and verification for issues we used to test! For example if there is audio at all or video, if so, is it good and proper or is it faulty? Does it have glitches, frame issues and other possible obstructions? IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence. A test that captures a 0-byte mp4 (Bug #24) and PASSES because "the recording file exists" is the exact PASS-bluff pattern §11.4 forbids. Content-quality analysis is what closes that gap.

Audio quality analysis — every audio test that PASSES MUST verify ALL of: (1) **Presence** — non-trivial RMS amplitude in captured WAV / `/proc/asound/.../pcm*p/sub0/hw_params`. (2) **Channel count** — `ffprobe -show_streams` matches the test's claim (2.0 / 5.1 / 7.1). (3) **Sample rate + bit depth** — match the codec / pipeline under test. (4) **Glitch census** — XRUN / FastMixer underrun-overflow-partial / AudioFlinger `writeError` counts above tolerance MUST classify explicitly (PASS within budget, WARN above, FAIL on hard limits per §11.4.1 SKIP-vs-FAIL decision tree). (5) **Coexistence-artifact census** — for tests that exercise WiFi/BT alongside audio: BT TX queue overflow, A2DP src underflow, coex notification storms, 2.4 GHz radio contention.

Video quality analysis — every video test that PASSES MUST verify ALL of: (1) **Presence** — captured screen recording has non-zero file size AND `ffprobe -count_frames` reports decoded-frame total > 0. 0-byte mp4 (Bug #24) is the canonical PASS-bluff and triggers §11.4.4 STOP. (2) **Routing target** — analyzer + action-timeline confirms video appeared on the *intended* display (primary vs secondary HDMI; Bug #13 pattern). (3) **Frame health** — drop count, frame-time variance (jitter), freeze detection (SSIM > 0.99 for ≥ 1 s), tearing. (4) **Obstruction census** — Tesseract OCR scan for hostile overlays (Application not responding, Force close, sign-in dialog, geo-restriction overlay, ad break, paywall, App is not certified). (5) **Resolution + codec** — captured frame dimensions match the test's claim; downgrade is a PASS-bluff.

Challenges (HelixQA) are bound equally — every Challenge that asserts PASS MUST run all five audio + five video layers. A Challenge that scores PASS without applicable analysis is the same class of defect as a unit test that does.

Tooling guarantee: audio = tinycap + aplay --dump-hw-params + ffprobe + /proc/asound/parsers (lib/audio_validation.sh per §11.2.5). Video = screenrecord + ffprobe -count_frames + recording-analyzer + Tesseract OCR (scripts/dual_display_record.sh

- cmd/recording-analyzer/ per §11.4.2.A and §11.4.2.C). Tests dispatched against video evidence MUST honor §11.4.4 test-interrupt-on-discovery when the analyzer reports empty input — do not silently absorb that as a generic PASS-bluff banner.

Non-compliance is a release blocker regardless of context.

§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)

Forensic anchor — direct user mandate (verbatim, 2026-05-08T18:30 MSK):

"'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainty is allowed at any cost! We MUST always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!"

Tests, gates, status reports, closure narratives, commit messages, and operator-facing text MUST NOT use likely, probably, maybe, might, possibly, presumably, seems, or appears to when describing causes of failures, behaviour, or fix effectiveness. Either prove the cause with captured forensic evidence (logcat, dmesg, /sys readings, getprop, kernel ramoops, dropbox, strace, etc.) and state it as fact, OR explicitly mark UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: with a tracked-task ID for follow-up.

Pre-build gate CM-N0-GUESSING-MANDATE greps recently-modified docs

- test scripts for the forbidden vocabulary outside explicit UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: blocks. Paired mutation introduces a likely token into a fresh status block → gate FAILs. Propagation gate CM-COVENANT-114-6-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.6.

Non-compliance is a release blocker regardless of context.

§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)

A demotion from any FAIL classification (OPEN, POSSIBLE PRODUCT DEFECT, FAIL) to a lower-severity classification (INVESTIGATED, MITIGATED, RESOLVED, WORKING-AS-INTENDED) requires positive evidence captured under the **same conditions** that originally exposed the defect — same device, same firmware, same cycle position, same load profile.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged UNCONFIRMED: until same-conditions retest produces positive evidence. The expanded forbidden-vocabulary list:

Forbidden phrase	Why it bluffs
"isolated re-run PASSes therefore X was a flake"	Strips the very environment that exposed the defect.
"runtime drift"	Label for "we don't know what changed".
"intermittent" / "transient"	Label for "we don't know how to reproduce".
"pending stress retest"	Defers the actual investigation indefinitely.
"correlates with X"	Hypothesis presented as causation.

Pre-build gate CM-DEMOTION-EVIDENCE-RULE scans Issues.md / Fixed.md / CONTINUATION.md for these phrases outside explicit UNCONFIRMED: / UNATTRIBUTED: / PENDING_CYCLE_RETEST: blocks. Propagation gate CM-COVENANT-114-7-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.7.

Non-compliance is a release blocker regardless of context.

§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)

Before designing a non-trivial fix, implementing a new feature, or declaring an architectural choice, perform deep web research to verify the chosen approach is informed by current state-of-the-art. Research surface: official documentation (Android/AOSP/Khronos/CEA-861/AES/IEEE/IETF/ITU), vendor technical guides (Rockchip, Sipeed, Audinate Dante, Synaptics, Realtek, Bluetooth SIG), open-source codebases (Linux kernel, ALSA, Bluez, ExoPlayer, libVLC, MPV, FFmpeg, AOSP forks), coding tutorials + technical articles (Stack Overflow, AOSP Code Lab, AES papers), issue trackers (Android bug tracker, AOSP Gerrit, GitHub issues).

A fix that re-invents a wheel — or reproduces a known-broken pattern — when the open-source community has already solved the problem is a §11.4 violation by omission. Every non-trivial fix's commit / Issues.md / Fixed.md entry MUST cite at least one external source URL OR the literal "NO external solution found — original work".

Pre-build gate CM-RESEARCH-CITATION-PRESENT scans new fix-direction blocks for the pattern. Propagation gate CM-COVENANT-114-8-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Documentation continuity requirement: every fix landed under §11.4.8 also adds to docs/guides/ a user-facing or developer-facing guide section where appropriate.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.8.

Non-compliance is a release blocker regardless of context.

§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)

When closing a multi-defect batch, all source-side fixes that DO NOT require runtime on-device validation to design MUST be landed BEFORE the next firmware rebuild. Anti-pattern eliminated: Fix A → rebuild → flash → cycle → fix B → rebuild → ... serializes 7-8 hours per fix instead of batching all into ONE build cycle. Operator time is the scarce resource.

Exceptions documented in commit message as REQUIRES_REBUILD: <reason>: kernel-5.10/ changes, atmosphere-*.sh boot-script side-effects, hardware/rockchip/ HAL behavior — each gates downstream state and requires firmware to validate.

Before declaring a batch "ready for rebuild": pre-build GREEN + meta-test GREEN + existing-device validations performed where possible + Issues.md/Fixed.md/CONTINUATION.md in sync (+ HTML/PDF exported) + §11.4.8 research citations all logged.

Propagation gate CM-COVENANT-114-9-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.9.

Non-compliance is a release blocker regardless of context.

§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)

All credentials, secrets, API tokens, passwords, phone numbers, OAuth tokens, signing keys MUST NEVER live in tracked files. Templates with placeholder values are allowed (.example suffix). Tests load credentials at runtime from scripts/testing/secrets/ (or per-submodule equivalent); operator-populated files are chmod 600, directory is chmod 700. .env, .env.*, *.env patterns + scripts/testing/secrets/* (with .example + README.md exception) git-ignored project-wide.

Test scripts MUST NEVER echo credentials to stdout/stderr/logcat. Screen- recording of sign-in flows MUST redact credential-bearing frames. Per-service file separation (.netflix.env, .disney.env, etc.) limits blast radius.

Forensic-rotation policy: suspected leak → rotate at provider, update local .env, audit captured artifacts. Pre-build gate CM-CREDENTIAL-LEAK-SCAN greps tracked files for entropy-suspicious password strings + known API-token formats. Propagation gate CM-COVENANT-114-10-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.10.

Non-compliance is a release blocker regardless of context.

§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)

Whenever an HDMI sink with a network-accessible introspection API is present (current example: Arvus H2-4D-273 at <http://192.168.4.185/>), the test suite MUST consume the sink's report as captured-evidence for every audio test asserting a codec / channel-count / passthrough mode. On-SoC HAL telemetry ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Reference: scripts/testing/lib/arvus_probe.sh, scripts/testing/arvus_probe.sh, docs/guides/ARVUS_HDMI_INTEGRATION.md. Pre-build gate CM-ARVUS-EVIDENCE-INTEGRATED (7 invariants) + paired mutation. No hardcoding (env: ARVUS_HOST etc.). Topology dispatch per §11.4.3 — sink unreachable → SKIP, never FAIL. Identity verification (MAC match) before consuming codec-state. Anti-stickiness post-stop. HelixQA Challenges bound equally.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.13. Integration reference: docs/guides/ARVUS_HDMI_INTEGRATION.md.

Non-compliance is a release blocker regardless of context.

§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)

Every test that issues `am start / cmd media_session play / MediaController.play` MUST issue matching `am force-stop / input keyevent KEYCODE_MEDIA_STOP` + register cleanup in EXIT trap. Verified via positive evidence (Arvus codec-state → N.E., `dumpsys media_session` shows no PLAYING for test app). `test_all_fixes.sh` post-test sanity check FAILs the just-completed test if it left orphan playback. HelixQA Challenges bound equally. No grace period — "next test will clean it up" is §11.4 PASS-bluff.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.14. Pre-build gates CM-TEST-PLAYBACK-CLEANUP + CM-COVENANT-114-14-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)

Every active item in `docs/Issues.md` carries a `**Status:**` line with one of six values: Queued, In progress, Ready for testing, In testing, Reopened, Fixed (→ `Fixed.md`). Status MUST be updated as the item progresses through its lifecycle. Fixed requires captured-evidence per §11.4.5 + migration to `Fixed.md`.

The auto-generated `docs/Issues_Summary.md` includes the Status column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 amendment).

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.15. Pre-build gates CM-ITEM-STATUS-TRACKING + CM-COVENANT-114-15-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)

Every active item in `docs/Issues.md` carries a `**Type:**` line with one of three values: Bug (product defect / regression / user-visible broken behaviour), Feature (new capability not previously offered to end users), Task (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The vocabulary is CLOSED — no other value is permitted.

The auto-generated docs/Issues_Summary.md includes the Type column. All three file types (.md, .html, .pdf) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 + §11.4.16 amendment).

Canonical authority: parent docs/guides/ATMOSPHERE_CONSTITUTION.md §11.4.16. Pre-build gates CM-ITEM-TYPE-TRACKING + CM-COVENANT-114-16-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between the batch's fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — complete retest is typically 12-48 hour elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composes with §11.4.4 (per-fix retest) — §11.4.37 is the additional final integrity check at RELEASE granularity. Composes with §11.4.7 — full-suite retest is the authoritative baseline for closures in the batch. No escape hatch — no --skip-full-retest or --quick-release flag exists.

Pre-build gate CM-FULL-SUITE-RETEST-MANDATE + paired mutation. Propagation gate CM-COVENANT-114-40-PROPAGATION enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: constitution submodule Constitution.md §11.4.37.

Non-compliance is a release blocker regardless of context.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, or equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order): (1) `git fetch --all --prune --tags` against every configured remote — capture output. (2) Integrate every divergent commit locally via `git rebase` (local is strict superset), `git merge` (independent additions both deserve preservation), or operator-confirmed cherry-pick (remote subset already present locally). (3) Audit: no conflict markers (`grep -rn '^<<<<<< \|^===== $|^>>>>>>'` returns empty), no silent file drops (`git diff --stat HEAD@{1} HEAD`), every previously-passing test still passes per §11.4.4 / §11.4.40 baseline, every captured-evidence artifact still validates. (4) `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless §9.2 sub-clause 6 explicitly authorises it for a remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation.

Two-gate composition with CONST-043 — §11.4.41 does NOT relax CONST-043's operator-approval requirement. Gate A (CONST-043): operator types explicit per-operation force-push authorisation. Gate B (§11.4.41): agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push. Both gates required.

Verification artefact — every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing 7 elements: (i) `git fetch` output, (ii) per-remote `HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration test suite delta (must show only expected changes), (vi) `--force-with-lease` push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Composes with §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation), §11.4.32 (post-pull validation — audit step's mechanical companion), §11.4.37 (fetch-before-edit — step 1 enforces it for force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence requirement).

No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes. Pre-build gate CM-COVENANT-114-41-PROPAGATION enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs. Gate CM-FORCE-PUSH-MERGE-FIRST walks docs/changelogs/<tag>.md "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Canonical authority: constitution submodule Constitution.md §11.4.41.

Non-compliance is a release blocker regardless of context.

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Forensic anchor — direct user mandate (verbatim):

"We had to restart this session 3rd time in a row! The system of the host stays with no RAM memory for some reason! First make sure that whatever we do through our procedures related to this project **MUST NOT** use more than 60% of total system memory! All processes **MUST** be able to function normally!"

The mandate. Project procedures **MUST NOT** use more than **60% of total system RAM** (HOST_SAFETY_MAX_MEM_PCT). The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

Three consecutive session-loss SIGKILLS on 2026-04-30 during 1.1.5-dev — every one happened while scripts/build.sh was running m -j5 AOSP. Each Soong/Ninja job peaks at ~5–8 GiB RSS; collective RSS overran the 60% envelope and the kernel OOM-killer escalated, taking down user@1000.service. **§12.1's pre-flight check (refusing to start if host already distressed) was not enough** — the missing piece was an active CONSTRAINT on heavy work itself.

Mandatory protections (rock-solid):

1. HOST_SAFETY_MAX_MEM_PCT defaults to 60 in scripts/lib/host_session_safety.sh.
2. HOST_SAFETY_BUDGET_GB is computed at source-time from $\text{MemTotal} \times \text{MAX_PCT}/100$.

3. `bounded_run` clamps `MemoryMax` down to the budget if the caller asks for more (cgroup-level enforcement via `systemd-run --user --scope -p MemoryMax=...`).
4. `host_safe_parallel_jobs` and `host_safe_build_jobs` return the `safe -j` count given an estimated per-job RSS, capped at `nproc`.
5. `scripts/build.sh` wraps `m -j` in `bounded_run`. If the build's collective RSS exceeds the budget, only the scope is OOM-killed; `user@<uid>.service` stays alive.

Captured-evidence enforcement. Pre-build gate `CM-MEMBUDGET-METATEST` locks all 7 invariants and fires every pre-build run.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits. Operators who need more headroom should reduce parallelism, close other workloads, or add RAM — NOT raise the percentage.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §12.6.

Non-compliance is a release blocker regardless of context. *Built with zero-bluff commitment. Every feature actually works.*

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Forensic anchor — direct user mandate (verbatim, 2026-05-24):

"Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix or improvement landed in this project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load ($N \geq 100$ iterations OR ≥ 30 s wall-clock; per-iteration latency p50/p95/p99 recorded) + concurrent contention ($N \geq 10$ parallel invocations; no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input; every boundary produces a categorised result, never an uncaught exception).

Chaos (closed-set, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call; categorised recovery) + network-fault injection (drop/delay/reorder;

category=network|upstream per §11.4.69) + input-corruption injection (corrupt .env / config / input file mid-test; detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion; refuse cleanly OR degrade gracefully — NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault; recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS cites a captured-evidence artefact path per §11.4.5 + §11.4.69 (per-iteration latency.json, categorised_errors.txt, state_delta_snapshot.json, recovery_trace.log). Helper library stress_chaos.sh provides ab_stress_run, ab_stress_concurrent, ab_chaos_kill_pid_during, ab_chaos_drop_network_during, ab_chaos_corrupt_file_during, ab_chaos_oom_pressure_during, ab_chaos_disk_full_during, each composing with ab_pass_with_evidence / ab_skip_with_reason. Chaos-injection cleanup is non-negotiable — corrupt-restore, disk-fill-cleanup, process-restart MUST run in trap '...' EXIT; cleanup failure = §11.4.14 violation.

4-layer coverage per §11.4.4(b): pre-build gate (stress + chaos test files exist + executable + parseable under sh -n + bash -n per §11.4.67; helper library exists; the fix's pre-build gate cites the stress + chaos test file path) + paired meta-test mutation per §1.1 (stripping chaos-injection or per-iteration evidence capture → gate FAILs) + on-device test (if LIVE_ADB_TESTABLE per §11.4.51, dispatched against real device, evidence under qa-results/<run-id>/stress_chaos/) + HelixQA Challenge entry (if user-visible feature per §11.4.4(b) layer 4).

Composes with §11.4 / §107 (resilience IS end-user quality), §11.4.1 (FAIL-bluffs forbidden), §11.4.5 (captured-evidence quality applies to latency distribution + error categories), §11.4.6 (no guessing — categorised errors only), §11.4.43 (TDD RED-first under load/chaos), §11.4.50 (N iterations identical exit + identical evidence-hashes), §11.4.52 (autonomous validation), §11.4.69 (universal sink-side positive-evidence taxonomy), §11.4.83 (recovery transcripts ARE end-user-channel proofs).

Canonical authority: constitution submodule Constitution.md §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no --skip-stress, --no-chaos, --happy-path-suffices, --stress-test-later flag exists.